Rheinland-Pfälzische
Technische Universität
Kaiserslautern
Landau

Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#

Bachelor's Thesis

by

Jakob Meyerhöfer

March 18, 2025

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Sebastian Schloßer, M.Sc. (Sebastian Schweizer)

Declaration of Authorship

I hereby declare that the work I have submitted with the title „Optimizing Error Messages: The Impact of Optimized Error Messages in Teaching F#“ was independently written by me. I have fully disclosed all sources and aids used and have identified as quotations all passages of the work — including tables and figures — taken verbatim or in substance from other works or the internet, indicating the respective sources.

Kaiserslautern, on 18.3.2025

Jakob Meyerhöfer

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Summary

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Contents

1. Introduction **1**
- 1.1. Research Problem and Objectives . . . 2
- 1.2. Research Questions . . . 3
- 1.3. Thesis Structure . . . 3

2. Related Work **5**
- 2.1. Challenges for Novice Programmers . . . 5
- 2.2. Role of Error Messages in Learning . . . 5
- 2.3. Existing Approaches to Improving Error Messages . . . 5
- 2.4. Educational Feedback Theories . . . 5

3. Analysis of Previous Semester Data **7**
- 3.1. Data Collection and Methodology . . . 7
- 3.2. Compilation Success Rates . . . 8
- 3.3. Test Pass Rates . . . 8
- 3.4. Insights and Patterns . . . 9

4. Problem Analysis: Issues with Existing Error Messages **15**
- 4.1. Examples of Current Error Messages . . . 15
  - 4.1.1. Example 1 — `digitSum` . . . 16
  - 4.1.2. Example 2 — `sortedDigits` . . . 19
- 4.2. Information Gaps in Error Outputs . . . 22
- 4.3. Challenges for Novices in Deriving Test Information . . . 22

5. Assertify: A Library for Improving Error Messages **23**
- 5.1. Objectives and Design of Assertify . . . 23
- 5.2. Implementation Details and Wrappers . . . 23
- 5.3. Error Message Improvements (Before and After Comparisons) . . . 24
  - 5.3.1. Example 1 — `digitSum` . . . 25
  - 5.3.2. Example 2 — `sortedDigits` . . . 26
- 5.4. Usability and Design Considerations . . . 26

6. TODO: Further Methodology (unclear what comes next) **29**

7. Evaluation and Results **31**
- 7.1. Evaluation Methodology . . . 31
- 7.2. Metrics for Success . . . 31
- 7.3. Quantitative Analysis . . . 31
- 7.4. Qualitative Insights . . . 31
- 7.5. Comparison to Previous Results . . . 32

8. Conclusion **33**
8.1. Summary of Contributions 33
8.2. Key Findings 33
8.3. Interpretation of Results 33
8.4. Strengths and Limitations of Approach 33
8.5. Recommendations for Future Development 34

List of Figures **35**

List of Listings **37**

List of Tables **39**

A. My Code **41**

B. My Ideas **43**

Bibliography **45**

1. Introduction

All programmers have to start at the beginning: What even is programming? What do you need to start writing code? What are variables/methods/etc. and how do you use them? Then there are certain key concepts/algorithms and further down the road data structures like maps, sets and so on. But that is a long way to go from learning the very basics to independently writing your own programs in the end, including the implementations and the according unit tests to verify said implementations.

Understanding and interpreting error messages is a fundamental challenge for novice programmers taking their first steps. Error messages serve as an essential form of feedback, helping programmers diagnose issues in their code. However, standard error messages, particularly those generated by compilers and testing frameworks, are often cryptic, overly technical, and difficult for beginners to decipher. Especially when students are provided with templates that contain tests and the goal is to implement various methods or data structures that can then be tested via the provided tests. They are often not particularly familiar with unit testing and are now confronted with stacktraces and various formats of error output depending on the programming language and testing framework.

In F# there are various namespaces or testing frameworks available. A commonly used namespace is Microsoft.VisualStudio.TestTools.UnitTesting. For example by using the `Assert.AreEqual` method one might get the following output:

```
path\to\Tests.fs(68): error TESTERROR:
    a) digitSum Random (75ms): Error Message: Test method Tests+Tests.a) digitSum Random threw exception:
    System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (866870754, 297439050)):
    Original:
    9N
    Shrunk:
    0N
    with exception:
    Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException: Assert.AreEqual failed. Expected:<0>. Actual:<1>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.ThrowAssertFailed(String assertionName, String message) in /_/src/TestFramework/TestFramework/Assertions/Assert.cs:line 60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T](T expected, T actual, IEqualityComparer`1 comparer, String message, Object[] parameters) in /_/src/TestFramework/TestFramework/Assertions/Assert.AreEqual.cs:line 201
```

```
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
        ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:
        ↪  line 69
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

    Stack Trace:
        at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
        ↪  1448.Invoke(String message)
        at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
        ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 1.1. Example Ouput of `Assert.AreEqual`

For advanced programmers this does not look too cryptic or misleading, because they are used to extract the necessary information they need from this. Working as a tutor for a first semester lecture, I frequently observe students struggling with these messages, unsure of what went wrong and how to fix their code. Students with none to some programming experience alike. Instead of guiding them toward a solution, poorly structured error messages often lead to frustration and trial-and-error debugging. This hinders the learning process and increases the time required to grasp key programming concepts.

To address this issue, this thesis explores how improving error messages can enhance the learning experience for novice programmers. By providing clearer, more structured feedback, the aim is to help students understand their mistakes more effectively, ultimately leading to a more efficient and engaging learning process.

1.1. Research Problem and Objectives

The primary problem this thesis addresses is the difficulty novice programmers face when interpreting error messages. Standard compiler and testing framework messages often assume prior knowledge, making them inaccessible to beginners. This can lead to confusion, incorrect assumptions about the problem, and inefficient debugging strategies. The objectives of this research are as follows:

- Analyze the impact of traditional error messages on the learning process.
- Develop a modified approach that enhances the clarity and usefulness of error messages.
- Evaluate whether improved error messages lead to faster learning and better debugging strategies among students.

To achieve these objectives, I have developed a test class that extends standard error reporting by adding a compact summary of test results. This modification aims to provide clearer feedback, helping students identify and resolve issues more efficiently.

1.2. Research Questions

This thesis seeks to answer the following key research questions:

- How do traditional error messages impact the learning process of novice programmers?
- What specific issues do students encounter when interpreting standard error messages?
- Can modified error messages improve comprehension and debugging efficiency?
- How does structured feedback affect students' ability to correct their mistakes?

By addressing these questions, this research aims to provide insights into the effectiveness of improved error reporting and its potential role in programming education.

1.3. Thesis Structure

This thesis is structured as follows:

- **Chapter 2: Related Work** – Reviews existing research on challenges faced by novice programmers, the role of error messages in learning, and previous attempts at improving error feedback.
- **Chapter 3: Analysis of Previous Semester Data** – Presents an analysis of past student submissions, including compilation success rates and test pass rates, to identify common issues.
- **Chapter 4: Problem Analysis** – Examines specific problems with existing error messages, including real-world examples and gaps in the information they provide.
- **Chapter 5: Assertify — A Library for Improving Error Messages** – Introduces and explains the implementation of Assertify, a tool designed to enhance error feedback.
- **Chapter 6: Further Methodology (VS Code Extension?)** – TODO
- **Chapter 7: Evaluation and Results** – Presents an analysis of the impact of modified error messages, including both quantitative and qualitative findings.

- **Chapter 8: Conclusion** – Summarizes key findings, discusses their implications, and provides recommendations for future work.

This structure ensures a logical flow, moving from problem identification to solution development and evaluation, ultimately contributing to a better understanding of how error messages affect novice programmers.

2. Related Work

TODO

Apparently there is something to do.

[Tol12]

2.1. Challenges for Novice Programmers

TODO

Describe common difficulties novices face when learning programming.

2.2. Role of Error Messages in Learning

TODO

Discuss how error messages impact the debugging and learning process.

2.3. Existing Approaches to Improving Error Messages

TODO

Review current solutions and their limitations.

2.4. Educational Feedback Theories

TODO

Summarize relevant feedback theories from educational research.

3. Analysis of Previous Semester Data

In this chapter I will go over the data I analyzed: What data, how it was collected and so on.

3.1. Data Collection and Methodology

The data set I analyzed was collected during winter semester 2023/2024 in the module Concepts of Programming. The data was anonymized and forwarded to me by my supervisor. There were 220 students in total submitting their files individually. During the semester the students had to complete 12 sheets with a total of 16 relevant tasks. *Relevant* in this case means that these were coding tasks. The students were provided with a template project containing the following:

- A file called `Mini.fs` – a small library of useful methods and basic types that are used throughout all of the assignments. For instance `Mini.fs` simplifies the user input and output to shift the focus away from all possible edge cases of I/O this early on. Mostly used is the type `Nat`, describing a natural number including zero ($\mathbb{N}_0$), written as a non-negative integer with postfix '`N`': `0N`, `1N`, ..., `1234N`, etc.
- Further assignment-specific types (e.g. type definition of a linked list and the individual nodes)
- All relevant methods with their method signature and a placeholder `failwith "TODO"` for the body

They did not have access to the tests, but they were able to execute them via ExClaim by simply pressing a button to verify their solutions individually. Through this in particular I had access to all submissions for a single assignment to analyze the progress of each student.

To illustrate, consider a method that returns the minimum of 3 natural numbers. The template for this method might look as follows:

Template

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    failwith "TODO"
```

The method is implemented with the placeholder `failwith "TODO"` to ensure compilation even if not all methods are implemented so that they are still able to test their solution at any point. The students are supposed to replace this `failwith "TODO"` with their own implementation of that method.

Sample Solution

```
let minimumOfThreeNumbers (a: Nat) (b: Nat) (c: Nat): Nat =
    min (min a b) c
```

In this sample solution the placeholder was replaced by a compact solution for the assignment. Student submissions may very well contain `match`-statements, `if-else`-statements or similar structures, which are also considered to be valid solutions.

With this data I reconstructed every single submission snapshot, i.e. which files did a student submit when tests were executed. Then I used Docker containers to collect the results of the build (Command: `dotnet build`) and test process (Command: `dotnet test`) for further analysis. The results of that will be elaborated in the following sections.

3.2. Compilation Success Rates

TODO

Present the success rates for project compilation.

3.3. Test Pass Rates

TODO

Analyze the percentage of passed tests.

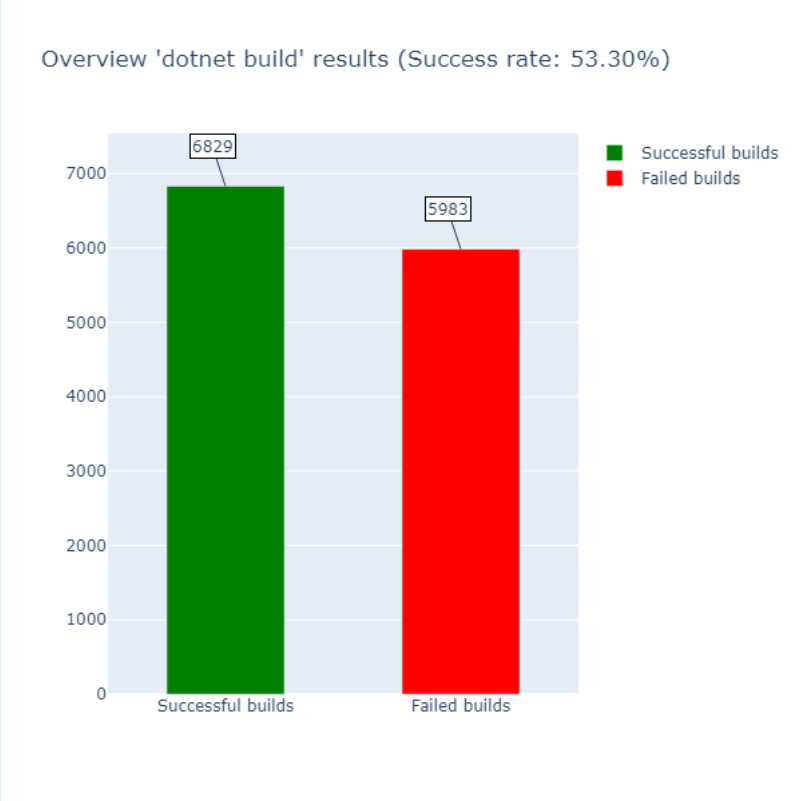

Figure 3.1. Build Stability Analysis: Success and failure counts for .NET builds, with overall success percentage.

3.4. Insights and Patterns

TODO

Highlight key patterns and findings from the data.

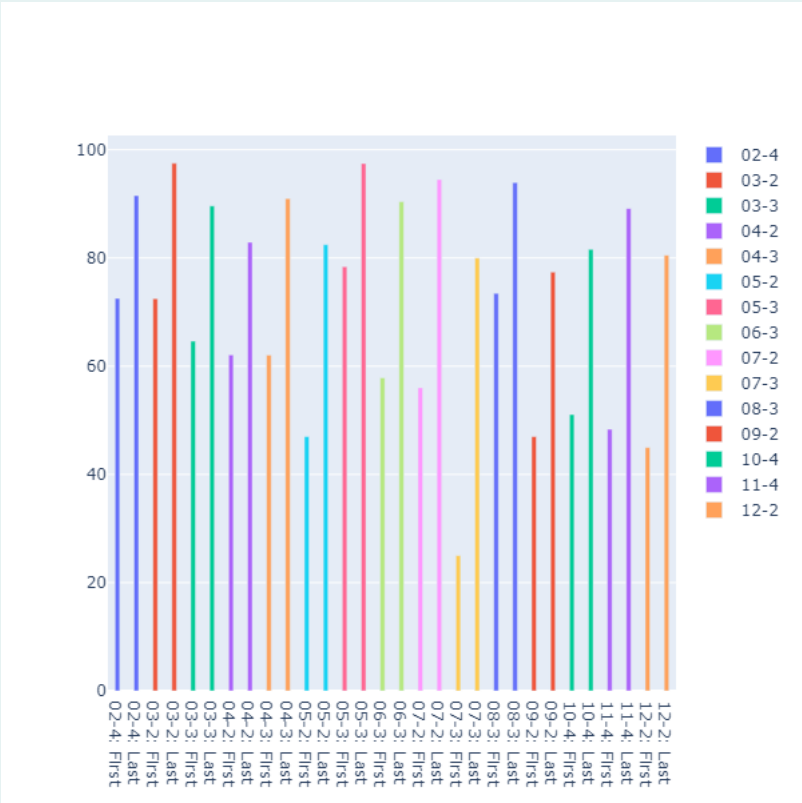

Figure 3.2. Submission Performance Analysis: Success rate of initial and final submissions across different exercises.

Pos	Error code	Code example with resulting error message
1st	FS0001	`let x: int = true` error FS0001: This expression was expected to have type 'int' but here has type 'bool'.

2nd	FS0058	`let f x = if x then 1 else`
		`-1`
		error FS0058: Unexpected syntax or possible incorrect indentation: this token is offside of context started at position (1:11). Try indenting this further. To continue using non-conforming indentation, pass the '-strict-indentation-' flag to the compiler, or set the language version to F# 7.

3rd	FS0588	`let add a b =`
		`  let result = a + b`
		error FS0588: The block following this 'let' is unfinished. Every code block is an expression and must have a result. 'let' cannot be the final code element in a block. Consider giving this block an explicit result.

4th	FS0597	`printfn "%d" List.min([1; 2])` error FS0597: Successive arguments should be separated by spaces or tupled, and arguments involving function or method applications should be parenthesized.
5th	FS3524	`if 1 < 2 then 1 else` error FS3524: Expecting expression.

Table 3.1. The top 10 of F# error codes in all submissions (1st – 5th)

Pos	Error code	Code example with resulting error message

6th	FS3546	`type Ex = | A of int * int`
		`match A(1,2) with A (, y) -> y`
		--- 	
		`error FS3546: Expecting pattern.`

7th	FS0033	`type Ex<'a,'b> = {A:'a; B:'b}`
		`let ex: Ex<int> = {A=1; B=2}`
		--- 	
		`error FS0033: The type 'FSI_0002.Ex<_,_>'`
		`expects 2 type argument(s) but is given 1.`

8th	FS0800	`type Ex = {A: int; B: int}`
		`let ex = Ex`
		--- 	
		`error FS0800: Invalid use of a type name.`

9th	FS3156	`let add a b = a +`
		--- 	
		`error FS3156: Unexpected token '+' or`
		`incomplete expression.`

10th	FS0727	`type Ex = | A of int * int`
		`match A (1, 2) with A x y -> x`
		--- 	
		`error FS0727: This union case expects 2`
		`arguments in tupled form, but was given 0.`
		`The missing field arguments may be any of:`
		`int`
		`int`

Table 3.2. The top 10 of F# error codes in all submissions (6th – 10th)

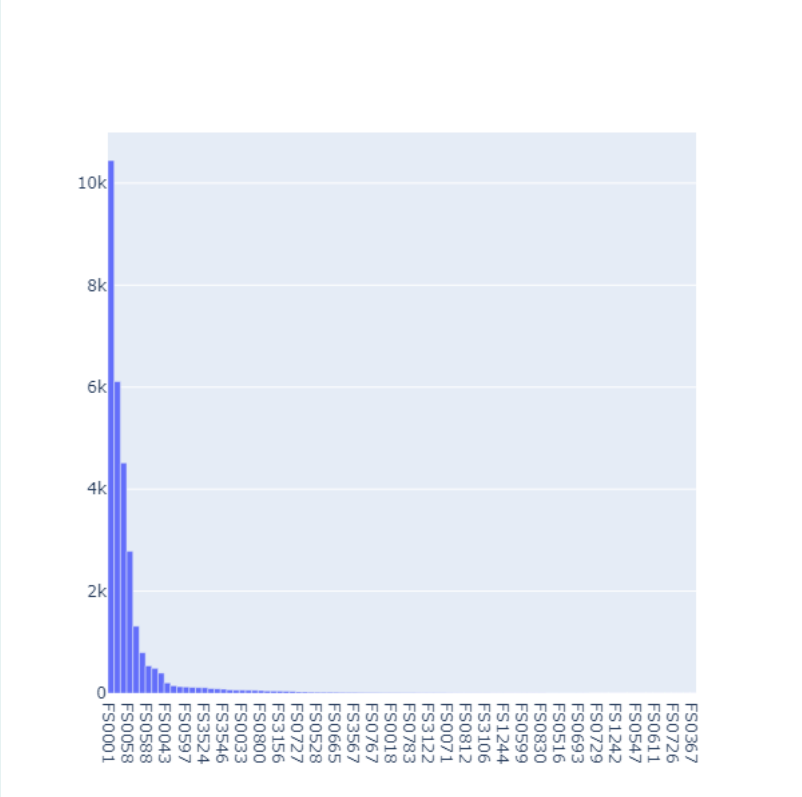

Figure 3.3. Error Code Analysis: Distribution and frequency of F#error codes across all exercises and submissions.

4. Problem Analysis: Issues with Existing Error Messages

TODO

Apparently there is something to do.

4.1. Examples of Current Error Messages

TODO

Show examples of unhelpful or ambiguous error messages.

4.1.1. Example 1 — `digitSum`

Implement a method `digitSum: Nat -> Nat`, that takes a natural number `n` as input and returns its digit sum. Examples:

```
digitSum 0N = 0N
digitSum 123N = 6N
digitSum 1234N = 10N
digitSum 4711N = 13N
```

```
let rec digitSum (n: Nat): Nat =
    if n = 0N then
        1N
    else
        (n % 10N) + digitSum (n / 10N)
```

Listing 4.1. Student submission for `digitSum`

```
let rec digitSumSolution (n: Nat): Nat =
    if n = 0N then
        0N
    else
        (n % 10N) + digitSumSolution (n / 10N)
```

Listing 4.2. Sample solution for `digitSum`

Example Tests — digitSum

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``a) digitSum Examples`` (): unit =
        test <@ digitSum 123N  = 6N @>
        test <@ digitSum 1234N = 10N @>
        test <@ digitSum 42N   = 6N @>
        test <@ digitSum 105N  = 6N @>
        test <@ digitSum 0N    = 0N @>
        test <@ digitSum 4711N = 13N @>
```

Listing 4.3. Example tests for digitSum

```
a) digitSum Examples (388ms): Error Message: Test method Tests+Tests.a)
↪  digitSum Examples threw exception:
    Swensen.Unquote.AssertionFailedException: Test failed:

    StudentSubmission.digitSum 123N = 6N
    StudentSubmission.digitSum 123N = 6N
    7N = 6N
    false

    Stack Trace:
        at Tests.Tests.a) digitSum Examples() in path\to\Tests.fs:line 57
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 4.4. Output of examples tests for digitSum

Key Information Example Tests — digitSum

Tested expression	digitSum 123N
Expected result	6N
Actual result	7N

Table 4.1. Key Information Example Tests — digitSum

Random Tests — digitSum

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``a) digitSum Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<Nat> (digitSumSolution n,
            ↪  digitSum n)
        )
```

Listing 4.5. Random tests for digitSum

```
a) digitSum Random (74ms): Error Message: Test method Tests+Tests.a)
↪  digitSum Random threw exception:
    System.Exception: Falsifiable, after 1 test (1 shrink) (StdGen (
    ↪  1765833463, 297438798)):
    Original:
    11N
    Shrunk:
    0N
    with exception:
    Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
    ↪  Assert.AreEqual failed. Expected:<0>. Actual:<1>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
        ↪  ThrowAssertFailed(String assertionName, String message) in /_/
        ↪  src/TestFramework/TestFramework/Assertions/Assert.cs:line 60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual, IEqualityComparer`1 comparer, String
        ↪  message, Object[] parameters) in /_/src/TestFramework/
        ↪  TestFramework/Assertions/Assert.AreEqual.cs:line 201
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
        ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
        ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.a) digitSum Random@69.Invoke(Nat n) in path\to\Tests.fs:
        ↪  line 69
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

    Stack Trace:
        at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
        ↪  1448.Invoke(String message)
        at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
        ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.a) digitSum Random() in path\to\Tests.fs:line 68
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj
        ↪  , BindingFlags invokeAttr)
```

Listing 4.6. Output of random tests for digitSum

Key Information Random Tests - digitSum

Tested expression	`a) digitSum Random` / `Shrunk: 0N` } `digitSum 0N`
Expected result	`Expected:<0>` $\Longrightarrow$ `0` (actually `0N`)
Actual result	`Actual:<1>` $\Longrightarrow$ `1` (actually `1N`)

Table 4.2. Title

4.1.2. Example 2 — `sortedDigits`

Implement a method `sortedDigits: Nat -> bool`, that takes a natural number `n` as input and returns `true` if the digits are in ascending order and `false` otherwise. Examples:

```
sortedDigits 0N = true          sortedDigits 1111N = true
sortedDigits 5N = true          sortedDigits 1101N = false
sortedDigits 159N = true        sortedDigits 543N = false
```

```
let rec sortedDigits (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigits (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit < lastDigit
```

Listing 4.7. Student submission for `sortedDigits`

```
let rec sortedDigitsSolution (n: Nat): bool =
    if n = 0N then
        true
    else
        let frontSorted: bool = sortedDigitsSolution (n / 10N)
        let lastDigit: Nat = n % 10N
        let secondLastDigit: Nat = (n / 10N) % 10N
        frontSorted && secondLastDigit <= lastDigit
```

Listing 4.8. Sample solution for `sortedDigits`

Example Tests — `sortedDigits`

```
    [<TestMethod>] [<Timeout(1000)>]
    member _.``b) sortedDigits Examples`` (): unit =
        test <@ sortedDigits 0N    = true @>
        test <@ sortedDigits 5N    = true @>
        test <@ sortedDigits 159N  = true @>
        test <@ sortedDigits 1111N = true @>
        test <@ sortedDigits 42N   = false @>
        test <@ sortedDigits 543N  = false @>
        test <@ sortedDigits 1101N = false @>
```

Listing 4.9. Example tests for `sortedDigits`

```
b) sortedDigits Examples (4ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Examples threw exception:
   Swensen.Unquote.AssertionFailedException: Test failed:

   StudentSubmission.sortedDigits 1111N = true
   StudentSubmission.sortedDigits 1111N = true
   false = true
   false

   Stack Trace:
        at Tests.Tests.b) sortedDigits Examples() in path\to\Tests.fs:line
        ↪   80
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪  arguments, Signature sig, Boolean isConstructor)
          at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object
          ↪  obj, BindingFlags invokeAttr)
```

Listing 4.10. Output of examples tests for `sortedDigits`

Key Information Example Tests — `sortedDigits`

Tested expression	`sortedDigits 123N`
Expected result	`true`
Actual result	`false`

Table 4.3. Title

Random Tests — `sortedDigits`

```
    [<TestMethod>] [<Timeout(5000)>]
    member _.``b) sortedDigits Random`` (): unit =
        Check.One (
            { Config.QuickThrowOnFailure with EndSize = 1000 },
            fun (n: Nat) -> Assert.AreEqual<bool> (sortedDigitsSolution n,
            ↪   sortedDigits n)
        )
```

Listing 4.11. Random tests for `sortedDigits`

```
b) sortedDigits Random (3ms): Error Message: Test method Tests+Tests.b)
↪  sortedDigits Random threw exception:
  System.Exception: Falsifiable, after 5 tests (0 shrinks) (StdGen (
  ↪  867569077, 297439050)):
  Original:
  33N
  with exception:
  Microsoft.VisualStudio.TestTools.UnitTesting.AssertFailedException:
  ↪  Assert.AreEqual failed. Expected:<True>. Actual:<False>.
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.
        ↪  ThrowAssertFailed(String assertionName, String message) in /
        ↪  _/src/TestFramework/TestFramework/Assertions/Assert.cs:line
        ↪  60
        at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[
        ↪  T](T expected, T actual, IEqualityComparer`1 comparer,
        ↪  String message, Object[] parameters) in /_/src/TestFramework
        ↪  /TestFramework/Assertions/Assert.AreEqual.cs:line 201
      at Microsoft.VisualStudio.TestTools.UnitTesting.Assert.AreEqual[T]
      ↪  (T expected, T actual) in /_/src/TestFramework/TestFramework/
      ↪  Assertions/Assert.AreEqual.cs:line 34
        at Tests.b) sortedDigits Random@90.Invoke(Nat n) in path\to\
        ↪  Tests.fs:line 90
        at FsCheck.Testable.evaluate[a,b](FSharpFunc`2 body, a a)

  Stack Trace:
      at Microsoft.FSharp.Core.PrintfModule.PrintFormatToStringThenFail@
      ↪  1448.Invoke(String message)
      at <StartupCode$FsCheck>.$Runner.get_throwingRunner@382.FsCheck.
      ↪  IRunner.OnFinished(String name, TestResult testResult)
        at FsCheck.Runner.runner[a](Config config, a prop)
        at FsCheck.Runner.check[a](Config config, a p)
        at FsCheck.Check.One[Testable](Config config, Testable property)
        at Tests.Tests.b) sortedDigits Random() in path\to\Tests.fs:line
        ↪   89
        at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
        ↪   arguments, Signature sig, Boolean isConstructor)
        at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object
        ↪  obj, BindingFlags invokeAttr)
```

Listing 4.12. Output of random tests for `sortedDigits`

Tested expression	`b) sortedDigits Random` / `Original: 33N` } `sortedDigits 33N`
Expected result	`Expected:<True>` $\Longrightarrow$ `True` (actually `true`)
Actual result	`Actual:<False>` $\Longrightarrow$ `False` (actually `false`)

Table 4.4. Title

Key Information Random Tests — `sortedDigits`

4.2. Information Gaps in Error Outputs

TODO

Analyze what critical information is missing in these messages.

4.3. Challenges for Novices in Deriving Test Information

TODO

Explain why deciphering test expectations is hard for beginners.

5. Assertify: A Library for Improving Error Messages

Assertify describes a library of various methods and classes to improve error messages during testing.

5.1. Objectives and Design of Assertify

Assertify incorporates functionalities to enhance the debugging and testing experience by providing clearer, more informative error messages. Traditional assertion failures often yield cryptic outputs, making it difficult to identify the root cause of an issue. Assertify addresses this by offering detailed, structured feedback, including expression simplifications, expected vs. actual values, and reduction history. The design principles behind Assertify focus on:

(1) **Clarity** – Displaying test failures in a structured and easily readable format.

(2) **Context** – Providing additional context through expression reductions and history tracking.

(3) **Usability** – Ensuring that error messages are helpful for students and developers alike.

(4) **Flexibility** – Supporting different levels of verbosity for various debugging needs.

By integrating these principles, Assertify streamlines the debugging process and improves the efficiency of error resolution.

5.2. Implementation Details and Wrappers

Assertify is implemented as a wrapper around standard test assertions, enriching the output with structured details. The core mechanism involves capturing the tested expressions, simplifying them where possible, and formatting the results into a well-structured report. Key implementation details include:

- **Expression Reduction:** Assertify simplifies complex expressions, stripping unnecessary details while preserving meaningful structure.
- **History Tracking:** The system logs successfully evaluated expressions leading up to a failure, providing insight into the progression of computations. This is especially helpful if a succession of operations are executed on the

same data structure and said data structure is correctly implemented for the majority of operations and only fails when a specific edge case occurs (e.g. an (almost) full linked list encounters a out-of-bounds index error). Then the history of operations might be helpful for novices to recreate the error because the order of operations is crutial to whether or not a operation fails.

- **Verbose and Compact Modes:** Users can toggle between detailed explanations and minimalistic outputs.
- **Enhanced Formatting:** Utilizing structured markers and color-coded elements, Assertify highlights critical differences between expected and actual results.

By wrapping around existing test frameworks, Assertify seamlessly integrates into existing workflows, enhancing error visibility without requiring significant changes to test code.

5.3. Error Message Improvements (Before and After Comparisons)

Traditional assertion failures often return extremely detailed messages that require additional effort to decipher. Assertify enhances this by providing detailed comparisons. The enhanced output not only shows expected vs. actual values but also presents the exact expression tested, making debugging significantly easier.

5.3.1. Example 1 — `digitSum`

TODO

Apparently there is something to do.

Example Tests — `digitSum`

TODO

(1) Compare old and new test results (old: 4.4.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.digitSum 123N
Simplified expression:      digitSum 123N
------------------- EXPECTED AND ACTUAL ----------------------
Expected result:            6N
Actual result:              7N
```

Listing 5.1. Assertify output of examples tests for `digitSum`

Random Tests — `digitSum`

TODO

(1) Compare old and new test results (old: 4.6.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.digitSum n
Simplified expression:      digitSum 0N
------------------- EXPECTED AND ACTUAL ----------------------
Expected result:            0N
Actual result:              1N
```

Listing 5.2. Assertify output of examples tests for `digitSum`

5.3.2. Example 2 — `sortedDigits`

TODO

Apparently there is something to do.

Example Tests — `sortedDigits`

TODO

(1) Compare old and new test results (old: 4.10.)

```
-------------------- TESTED EXPRESSION -------------------------
Tested/Failed expression:   StudentSubmission.sortedDigits 1111N
Simplified expression:      sortedDigits 1111N
------------------- EXPECTED AND ACTUAL ------------------------
Expected result:            true
Actual result:              false
```

Listing 5.3. Assertify output of examples tests for `sortedDigits`

Random Tests — `sortedDigits`

TODO

(1) Compare old and new test results (old: 4.12.)

```
-------------------- TESTED EXPRESSION -----------------------
Tested/Failed expression:   StudentSubmission.sortedDigits n
Simplified expression:      sortedDigits 33N
------------------- EXPECTED AND ACTUAL ---------------------
Expected result:            true
Actual result:              false
```

Listing 5.4. Assertify output of examples tests for `sortedDigits`

5.4. Usability and Design Considerations

Assertify is designed with a strong focus on usability, particularly for students learning programming and debugging. Key considerations include:

- **Readable Error Reports:** Formatting failures in a structured, natural-language format ensures accessibility.
- **Minimal Intrusion:** Assertify seamlessly integrates with existing test frameworks without requiring major changes.

- **Customizability:** Users can adjust verbosity levels and enable/disable history tracking based on their needs.
- **Color Coding and Annotations:** Highlighting key differences improves readability and makes debugging more intuitive.

These design choices collectively improve the debugging experience, reducing frustration and helping developers quickly understand and fix issues.

6. TODO: Further Methodology (unclear what comes next)

TODO

(1) VS Code Extension

(2) ExClaim Embedding

7. Evaluation and Results

TODO

Apparently there is something to do.

7.1. Evaluation Methodology

TODO

Explain how you assessed the effectiveness of Assertify.

7.2. Metrics for Success

TODO

Describe the criteria used to measure improvements.

7.3. Quantitative Analysis

TODO

Present the data and statistical results.

7.4. Qualitative Insights

TODO

Share insights from student feedback or observations.

7.5. Comparison to Previous Results

TODO

Highlight improvements compared to the baseline semester data.

8. Conclusion

TODO

Apparently there is something to do.

8.1. Summary of Contributions

TODO

Recap your key contributions.

8.2. Key Findings

TODO

Highlight the most important results and insights.

8.3. Interpretation of Results

TODO

Analyze what the results reveal about error messages and learning.

8.4. Strengths and Limitations of Approach

TODO

Identify the strengths and potential weaknesses of the approach.

8.5. Recommendations for Future Development

TODO

Suggest possible extensions or refinements.

List of Figures

3.1. Build Stability Analysis 9
3.2. Submission Performance Analysis 10
3.3. Error Code Analysis 13

List of Listings

1.1. Example Ouput of `Assert.AreEqual` 1

4.1. Student submission for `digitSum` 16
4.2. Sample solution for `digitSum` 16
4.3. Example tests for `digitSum` 17
4.4. Output of examples tests for `digitSum` 17
4.5. Random tests for `digitSum` 18
4.6. Output of random tests for `digitSum` 18
4.7. Student submission for `sortedDigits` 19
4.8. Sample solution for `sortedDigits` 19
4.9. Example tests for `sortedDigits` 20
4.10. Output of examples tests for `sortedDigits` 20
4.11. Random tests for `sortedDigits` 21
4.12. Output of random tests for `sortedDigits` 21

5.1. Assertify output of examples tests for `digitSum` 25
5.2. Assertify output of examples tests for `digitSum` 25
5.3. Assertify output of examples tests for `sortedDigits` 26
5.4. Assertify output of examples tests for `sortedDigits` 26

List of Tables

3.1. The top 10 of `F#` error codes in all submissions (1st – 5th) 11
3.2. The top 10 of `F#` error codes in all submissions (6th – 10th) . . . 12

4.1. Key Information Example Tests — `digitSum` 17
4.2. Title 19
4.3. Title 20
4.4. Title 22

A. My Code

TODO

Put detailed code here.

B. My Ideas

TODO

Ideas??

Bibliography

[Tol12] J.R.R. Tolkien. *The Lord of the Rings: One Volume*. The Lord of the Rings. Houghton Mifflin Harcourt, 2012. ISBN: 9780547951942.